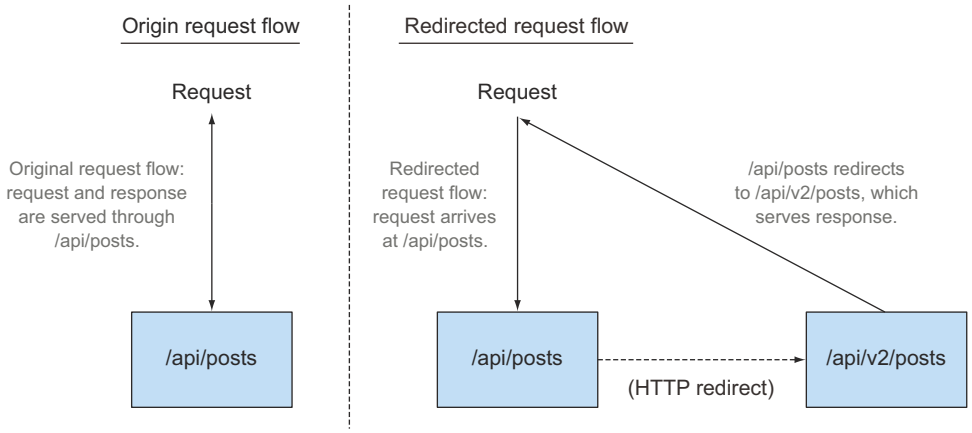


Figure 6.15 Sample server before and after introducing a redirect

6.8 CORS and redirects

When performing an HTTP redirect, both the server initiating the redirect and the server receiving the redirect must have the correct CORS headers. If either server doesn't have the correct CORS configuration, or doesn't support CORS at all, the request will fail.

As an API evolves, it may have the need to use HTTP redirects. While we like to think that our API design is perfect from the beginning, the truth is that APIs change based on user's needs and feedback, and it's sometimes necessary to create a whole new URL endpoint. In these cases, HTTP redirects give the server a mechanism for connecting the old URL to the new URL. Visitors to the old URL are seamlessly redirected to the new one. This ensures that old URLs will continue to work even as an API evolves.

Suppose you want to introduce a new API version living at /api/v2/posts. After a lot of testing and bug fixes, you decide that the API is ready for everyone to use, including users of the previous version. So you set up an HTTP redirect from /api/posts (the old endpoint) to /api/v2/posts (the new endpoint). Figure 6.15 shows what your new API would look like.

The following snippet shows the code that implements a redirect which consists of two parts: the HTTP status code and the Location header. The status code indicates that the response is a redirect. There are two valid status codes for redirects: 301, which indicates that a resource has permanently moved to a new URL, and 302, which indicates a temporary move. (An example of a temporary redirect is if a server is down and you want to redirect the visitor to a status page for the duration of the outage.) The Location header indicates where the new resource lives. The Location header can point to a resource on the same server, or on a completely different server.

```
serverapp.get('/api/posts', function(req, res) {
  res.redirect(301, '/api/v2/posts');
});
```